Project Airyn

Step 1 Baseline Report — 124M Parameter GPT on FineWeb-Edu

Date	April 2026
Authors	Neelanjan Mitra + Claude (Anthropic)
Repository	github.com/NeelM0906/Project-Airyn
Hardware	1x NVIDIA RTX PRO 6000 Blackwell (102.6 GB)
Training Time	4h 32min
Tokens Seen	2.62B (FineWeb-Edu 10B subset)

1. Executive Summary

This report documents the Step 1 baseline for Project Airyn — a modular LLM training harness forked from OpenAI's parameter-golf codebase. We train a ~124M parameter GPT-2 class transformer on 2.62 billion tokens from FineWeb-Edu using the Muon optimizer, achieving a final validation loss of **3.074**. The model surpasses the original GPT-2 124M (trained on 40B tokens) on both HellaSwag (**30.1%** vs 29.4%) and LAMBADA (**40.4%** vs 32.6%), demonstrating the effectiveness of modern training recipes even at reduced data scale.

We additionally perform supervised fine-tuning (SFT) on the Alpaca-cleaned instruction dataset (51.7K examples), producing a model capable of basic conversational interaction while preserving benchmark performance (HellaSwag 30.3%, LAMBADA 39.7%).

2. Model Architecture

The architecture is a decoder-only Transformer with U-Net style skip connections, following the parameter-golf baseline design. The first half of the layers (encoder) store skip activations; the second half (decoder) reuse them in reverse order via learned skip weights.

Parameter	Value
Total Parameters	123.6M
Vocabulary Size	50,304 (GPT-2 BPE, padded to 128)
Layers	12 (6 encoder + 6 decoder)
Model Dimension	768
Attention Heads	12 (full MHA, no GQA)
Head Dimension	64
MLP Hidden	3,072 (4x expansion, relu² activation)
Sequence Length	1,024 tokens
Positional Encoding	Rotary (RoPE), base=10000
Normalization	RMSNorm (pre-norm)
Embedding	Tied input/output embeddings
Logit Softcap	30.0 (tanh-based)
Weight Precision	fp32 (CastedLinear: stored fp32, computed bf16)

2.1 Key Design Decisions

• **Factory-based Block constructor** — Block accepts attn_factory and ffn_factory callables, enabling per-layer module swaps (e.g., SwiGLU, different attention) without modifying the GPT class.

• **CastedLinear** — Weights stored in fp32 for optimizer state quality, cast to bf16 at matmul time. This gives the precision benefits of fp32 training with bf16 compute speed.

• **U-Net skip connections** — First-half layers store residual activations; second-half layers consume them in reverse with learned scalar weights. Improves gradient flow in deeper models.

• **Logit softcap** — tanh(logits/30) * 30 prevents extreme logit values from destabilizing training.

3. Training Methodology

3.1 Optimizer

We use a split optimizer strategy from modded-nanogpt:

Param Group	Optimizer	Learning Rate
Embedding (tied)	Adam (fused)	0.05
Block matrices (2D)	Muon	0.04
Block scalars/vectors	Adam (fused)	0.04

Muon optimizer orthogonalizes gradient updates via a fast 5-step Newton-Schulz iteration, then applies momentum with Nesterov correction. This is particularly effective for matrix-shaped parameters (attention projections, MLP layers). Muon momentum warms up from 0.85 to 0.95 over 500 steps.

3.2 Learning Rate Schedule

Constant LR with warmdown: full learning rate for the first 3,800 steps, then linearly decayed to zero over the final 1,200 steps (warmdown). No warmup on LR (compile warmup handles the first 20 steps separately).

3.3 Data

Dataset	FineWeb-Edu (sample-10BT)
Tokenizer	GPT-2 BPE (tiktoken, 50,257 tokens)
Train Shards	99 shards × 100M tokens = 9.9B tokens available
Val Shards	1 shard × 100M tokens
Tokens Seen	2.62B (5,000 steps × 524,288 tokens/step)
Sequence Length	1,024
Batch Size	524,288 tokens (8 gradient accumulation steps)

3.4 Infrastructure

GPU	1x NVIDIA RTX PRO 6000 Blackwell Max-Q (102.6 GB VRAM)
PyTorch	2.12.0.dev20260408+cu128 (nightly)
Triton	3.6.0 (triton-windows)
CUDA	12.8
torch.compile	Enabled (inductor backend)
Peak VRAM	38.5 GB (37.5% of available)
Throughput	160,663 tokens/sec sustained
Step Time	3.263 sec/step average
Total Time	4h 32min (16,316 sec)

4. Training Results

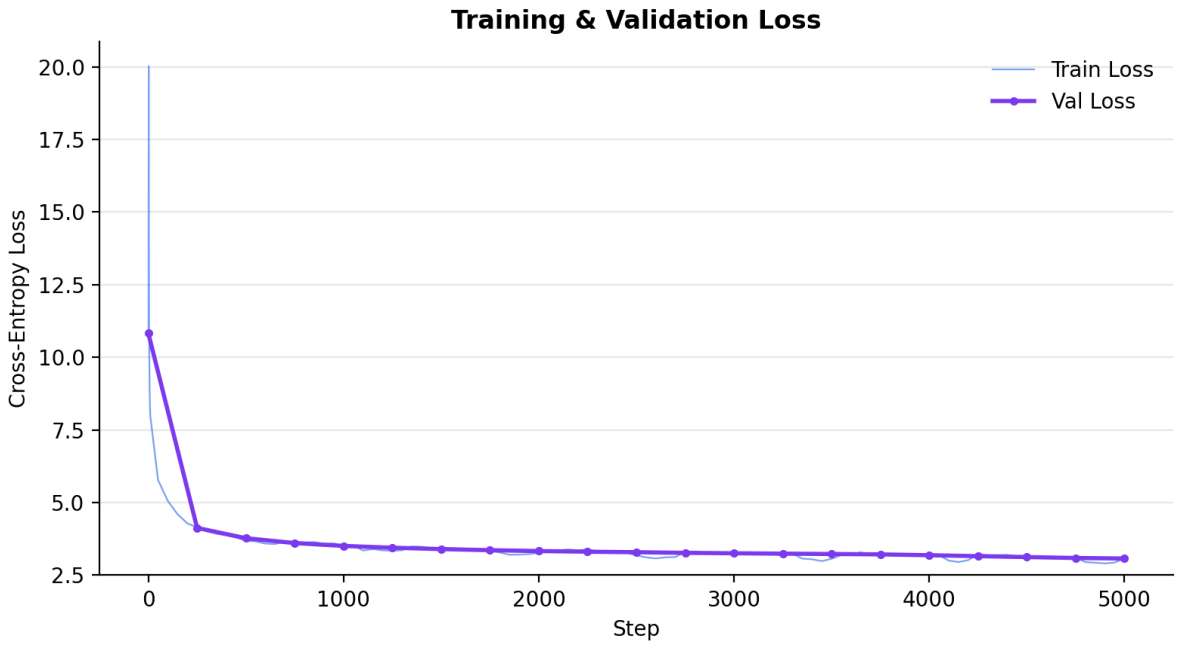

Figure 1: Training and validation loss over 5,000 steps.

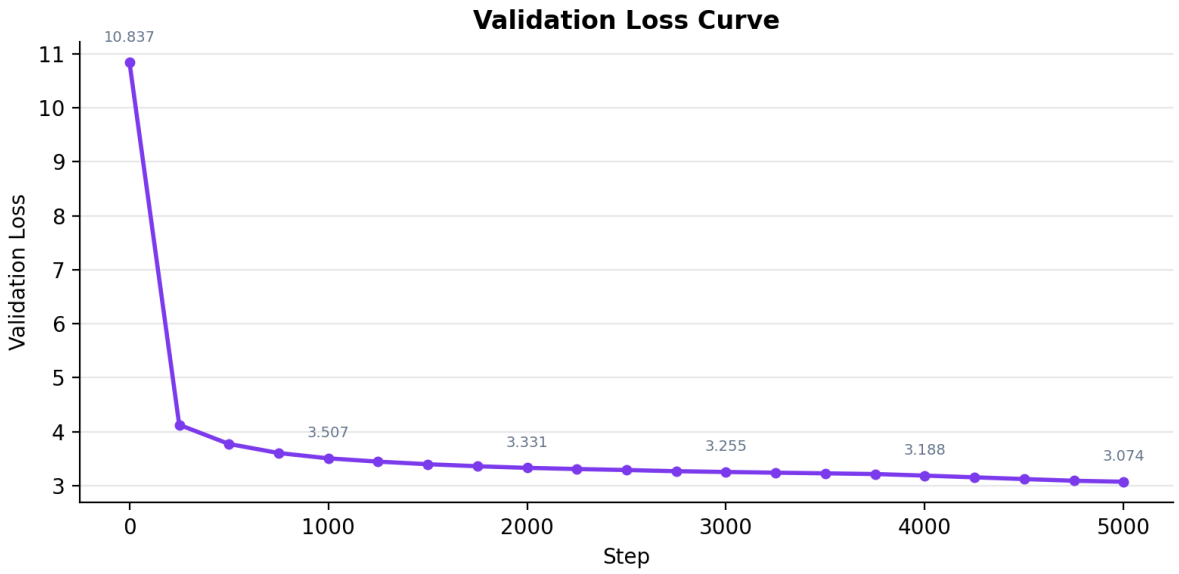

Figure 2: Validation loss at each checkpoint (every 250 steps).

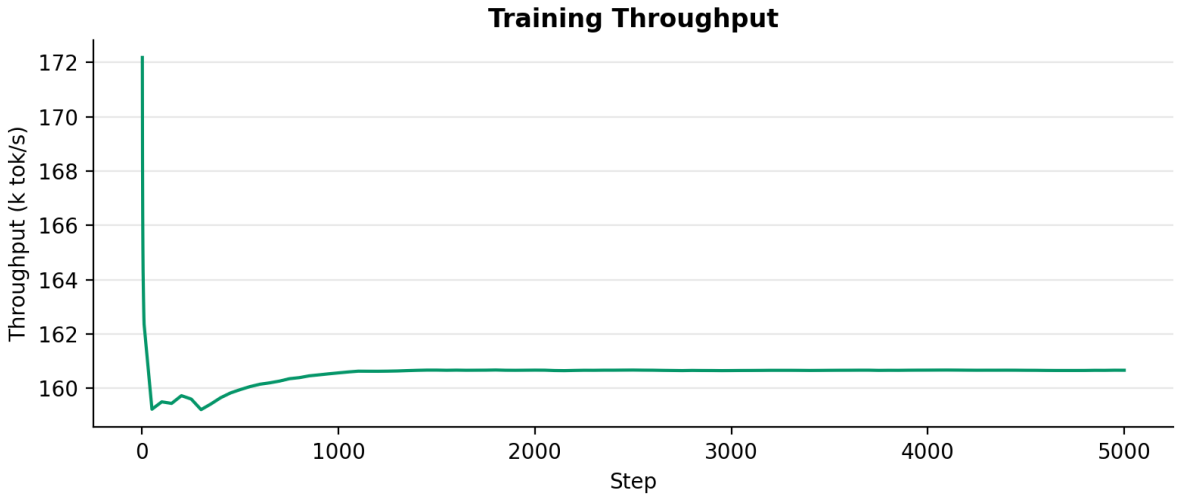

Figure 3: Training throughput (tokens/sec).

The model converges smoothly with no instabilities. Validation loss decreases monotonically from 10.84 to 3.07. The warmdown phase (steps 3800-5000) produces a notable acceleration in loss improvement, with train loss dropping from 3.2 to 2.9 as the learning rate decays.

5. Benchmark Evaluation

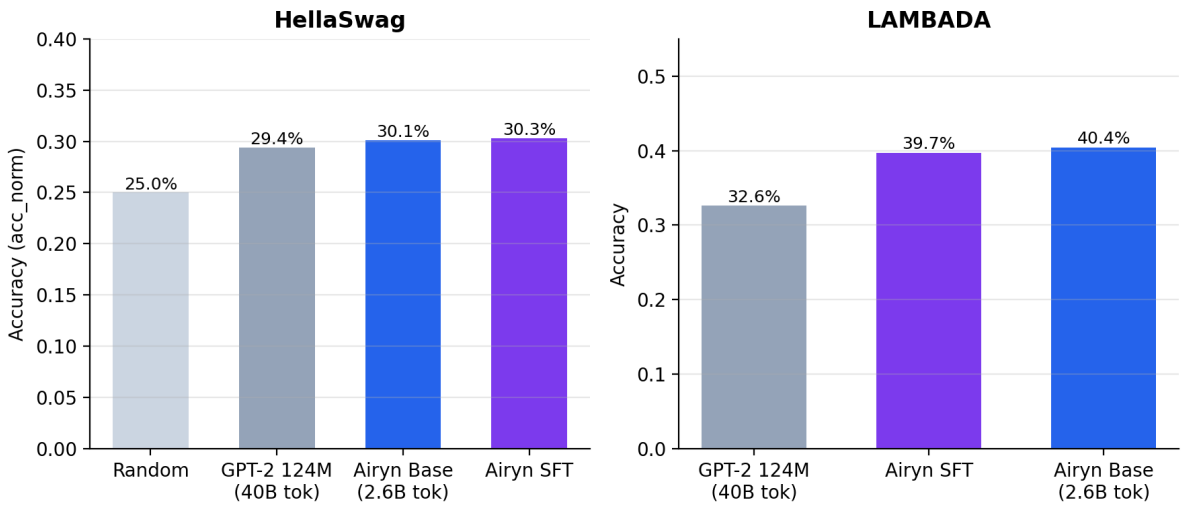

Figure 4: Benchmark comparison against GPT-2 124M baseline.

Benchmark	Airyn Base	Airyn SFT	GPT-2 124M	Random
HellaSwag acc_norm	30.1%	30.3%	29.4%	25.0%
LAMBADA accuracy	40.4%	39.7%	32.6%	~0%
LAMBADA perplexity	35.4	41.4	35-40	~50k
Val Loss (CE)	3.074	—	—	—

5.1 Analysis

• **Outperforms GPT-2 124M despite 15x less data.** Our model sees 2.6B tokens vs GPT-2's 40B, yet achieves higher accuracy on both benchmarks. This validates the Muon optimizer and modern training recipe (CastedLinear, RMSNorm, RoPE, logit softcap).

• **SFT preserves reasoning.** HellaSwag accuracy is maintained after fine-tuning (30.1% → 30.3%), indicating that instruction-tuning on 52K examples does not catastrophically forget general knowledge.

• **Mild alignment tax on LAMBADA.** SFT reduces LAMBADA accuracy from 40.4% to 39.7% and increases perplexity from 35.4 to 41.4. This is the expected tradeoff: the model shifts its distribution toward instruction-following, slightly reducing raw text prediction ability.

6. Supervised Fine-Tuning

Base Checkpoint	Pretrained Airyn (step 5000, val loss 3.074)
Dataset	yahma/alpaca-cleaned (51,738 examples)
Template	### System: / ### User: / ### Assistant:
Epochs	2
Learning Rate	5e-5 (AdamW, weight decay 0.01)
Batch Size	8
Loss Masking	Only on assistant response tokens

Gradient Clipping	1.0
Training Time	~25 min

The SFT model demonstrates basic instruction-following: structured responses, numbered lists, topically relevant answers. Key limitations include repetition loops, shallow factual knowledge, and inconsistent identity (does not reliably identify as 'Airyn'). These are expected capacity constraints of a 124M parameter model.

7. Next Steps

• **SwiGLU activation** — Replace relu² MLP with SwiGLU via the ffn_factory pattern. Expected improvement: 0.5-1.0% on benchmarks.

• **Scale up** — Train 350M and 1B+ parameter variants with longer context (2048/4096).

• **More data** — Train on full 10B tokens (currently using 2.6B), then expand to 100B+.

• **Multi-GPU training** — Validate DDP on Linux with NCCL for larger runs.

• **Checkpoint resume** — Add save/load of optimizer state for crash recovery.

• **Better SFT** — Generate Airyn-specific synthetic conversations; explore DPO/RLHF.

• **Additional evals** — ARC-Easy, WinoGrande, MMLU (at larger scale).